

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра информационных систем

Интеллектуальная система автоматизированной сверки вёрстки веб-страницы с макетом
Figma на основе компьютерного зрения и локальных языковых моделей

Курсовая работа по дисциплине
«Учебная практика (научно-исследовательская работа)»
09.03.02 Информационные системы и технологии
Информационные технологии в цифровом дизайне

Зав. Кафедрой _____ д. ф.-м. н, доцент С.Д. Махортов
Руководитель практики _____ ассистент В.А. Ушаков
Обучающийся _____ О. С. Петрыкина, 3 курс, д/о

Воронеж 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ГЛОССАРИЙ.....	4
ВВЕДЕНИЕ.....	5
1 Предметная область и существующие решения.....	7
1.1 Характеристика предметной области визуальной проверки вёрстки.....	7
1.2 Обзор и сравнительный анализ существующих решений.....	8
1.2.1 Облачные платформы визуального контроля.....	9
1.2.2 Открытые инструменты попиксельного сравнения.....	11
1.2.3 Ручная сверка.....	12
1.2.4 Сравнительный анализ и выводы.....	13
2 Проектирование системы.....	16
2.1 Постановка задачи на разработку.....	16
2.2.1 Функциональные требования.....	17
2.2.2. Нефункциональные требования.....	18
2.3 Архитектура системы.....	18
2.4. Проектирование модульной структуры и потоков данных.....	20
2.5 Проектирование форматов конфигурации, хранения артефактов и отчётности.....	23
2.6. Выводы по проектной части.....	24
3. Реализация разрабатываемой системы.....	26
3.1. Реализация конвейера получения и первичной обработки изображений.....	26
3.2. Реализация нейросетевых модулей классификации.....	28
и сопоставления фрагментов.....	28
3.3. Интеграция локальной языковой модели и генерация текстовых отчётов.....	30

3.4. Реализация веб-интерфейса управления и API взаимодействия.....	31
3.5. Выводы по реализационной части.....	34
3.6. Пример работы системы.....	36
ЗАКЛЮЧЕНИЕ.....	39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	40

ГЛОССАРИЙ

1. Diff-карта (карта различий) — изображение, получаемое в результате сравнения эталонного макета из Figma и актуального скриншота веб-страницы. На diff-карте обычно выделяются цветом области визуальных расхождений.

2. Fallback-вариант — резервный режим работы системы, который автоматически активируется при недоступности нейросетевых компонентов (например, PyTorch или Ollama). В этом режиме система переключается на классические алгоритмы компьютерного зрения и эвристический анализ.

3. Figma API — программный интерфейс сервиса Figma, позволяющий автоматически экспортировать дизайн-макеты в виде изображений (PNG) по идентификатору файла и узла.

4. Hotspots (горячие зоны) — области изображения с наибольшей плотностью визуальных различий, требующие первоочередного анализа.

5. Viewport — видимая область веб-страницы в окне браузера, определяемая его шириной и высотой. Размер viewport существенно влияет на расположение и внешний вид элементов интерфейса.

6. Визуальное регрессионное тестирование — автоматизированная проверка интерфейса путём сравнения скриншотов до и после внесения изменений в код с целью обнаружения непреднамеренных визуальных изменений (регрессий).

7. Локальная языковая модель (LLM) — большая языковая модель (в работе — Ollama), запущенная на локальном компьютере пользователя. Обеспечивает генерацию текстовых описаний дефектов без передачи данных во внешние сервисы.

8. Попиксельное сравнение — метод визуального тестирования, при котором два изображения сравниваются пиксель за пикселем для выявления различий.

ВВЕДЕНИЕ

Современная веб-разработка отличается высокой динамикой и итеративностью. Дизайн-макеты в Figma обновляются ежедневно, требования к пиксельной точности вёрстки постоянно растут, однако процесс визуальной приёмки по-прежнему остаётся преимущественно ручным. Тестировщики вынуждены вручную сравнивать макет и реализованную страницу, что приводит к субъективности оценки, пропуску визуальных регрессий и значительным временным затратам на каждом релизном цикле.

Как отмечает П. Попов, «скриншотные тесты — это способ проверки интерфейса приложения путём сравнения изображений» [1]. Наиболее распространённым методом остаётся попиксельное тестирование, при котором изображения сравниваются пиксель к пикселю, и багами будут являться даже минимальные отличия.

Существующие инструменты автоматизированного визуального тестирования (Percy, AppliTools, BackstopJS и др.) позволяют «захватывать изображения страниц или компонентов до и после изменений, а затем выявлять визуальные различия (пиксельные несоответствия)» [2]. Тем не менее, данные решения имеют ряд существенных ограничений: слабую интеграцию с Figma API, отсутствие семантического анализа выявленных отличий, сильную зависимость от облачной инфраструктуры и недостаточную локализацию отчётов.

Актуальность темы исследования определяется несколькими ключевыми факторами. Во-первых, сохраняется устойчивая проблема рассинхронизации между дизайном и фронтендом: «где-то используется не тот цвет, где-то не тот отступ, не то скругление, размер — и в итоге мы в реальности получаем вообще не такой интерфейс, как на макете» [3]. Во-вторых, возрастает потребность в эффективном визуальном регрессионном тестировании — «автоматизированной практике сравнения скриншотов UI приложения до и после изменений кода для обнаружения непредвиденных визуальных различий» [4]. В-третьих, развитие технологий локальных больших языковых моделей, в частности платформы Ollama, которая «предназначена для запуска больших языковых моделей

локально, на вашем компьютере» без передачи данных во внешние облачные сервисы [5], открывает новые возможности для создания интеллектуальных систем визуальной валидации.

Объектом исследования является процесс визуальной сверки вёрстки веб-страницы с утверждённым дизайн-макетом в среде Figma.

Предметом исследования выступают методы и программные средства построения конвейера автоматизированной визуальной валидации на основе компьютерного зрения и локальных языковых моделей.

Цель работы — разработать программную систему для автоматизированного сравнения вёрстки с макетом Figma, выявления визуальных расхождений и формирования структурированного отчёта о дефектах с применением методов машинного обучения, а также продемонстрировать работоспособность системы на реальном примере сравнения вёрстки с макетом Figma.

1 Предметная область и существующие решения

1.1 Характеристика предметной области визуальной проверки вёрстки

Предметная область исследования — процесс визуальной проверки соответствия реализованной вёрстки веб-страницы утверждённому дизайн-макету. Данная задача возникает на заключительном этапе разработки интерфейса и направлена на обеспечение максимального соответствия между задуманным дизайном и конечной реализацией.

Исходными данными для анализа выступают два ключевых артефакта. Эталон — экспортированное изображение (PNG) дизайн-макета из Figma. Как отмечают специалисты VK Tech, «мы забираем данные напрямую из Figma-файла через API» [6], что позволяет автоматизировать получение эталонных изображений по идентификаторам файла и узла. Факт — актуальный скриншот веб-страницы, сделанный в браузере при заданных размерах viewport.

Процесс сравнения включает несколько этапов: выравнивание эталонного и фактического изображения, попиксельное сравнение и классификацию выявленных отличий по степени значимости.

Завершающим этапом работы системы является формирование детализированного отчёта, содержащего подсвеченные проблемные зоны на изображениях и понятные описания выявленных дефектов. В рамках исследования применяется гибридный подход, сочетающий классические методы компьютерного зрения с возможностями локальных языковых моделей для генерации понятных текстовых описаний. По аналогии с конвейером обработки изображений в компьютерном зрении, где стадии будут реализованы» последовательно в едином пайплайне , в работе построена модульная цепочка анализа .

1.2 Обзор и сравнительный анализ существующих решений

В ходе анализа существующих подходов к визуальной проверке веб-интерфейсов были рассмотрены три основных класса решений: коммерческие облачные платформы, открытые инструменты попиксельного сравнения и традиционная ручная сверка. Облачные сервисы (Percy, AppliTools, Chromatic) обеспечивают стабильную инфраструктуру и встроенный анализ, однако требуют платной подписки, передачи скриншотов на внешние серверы и формируют отчёты исключительно на английском языке, что ограничивает их применение в проектах с повышенными требованиями к конфиденциальности и локализации. Открытые решения (BackstopJS, встроенные средства Playwright) позволяют выполнять сравнение локально и бесплатно, но отличаются высокой чувствительностью к техническому шуму рендеринга, отсутствием интеграции с Figma API и неспособностью генерировать семантические описания дефектов. Ручная проверка, несмотря на нулевую стоимость и интуитивность, подвержена субъективному фактору, не масштабируется при частых релизах и не формирует верифицируемых отчётов.

Проведённое сравнение выявило отсутствие на рынке инструмента, сочетающего полностью локальное выполнение, прямую интеграцию с дизайн-системой Figma, интеллектуальную фильтрацию ложных срабатываний и генерацию структурированных русскоязычных отчётов. Разрабатываемая система закрывает данный пробел за счёт гибридной архитектуры: классические алгоритмы компьютерного зрения обеспечивают точное выявление расхождений, нейросетевые модули (TinyDiffCNN, MultiAspectComparator, InterferenceAwareMatcher) классифицируют и фильтруют артефакты, а локальная языковая модель (Ollama) трансформирует технические данные в человекочитаемые баг-репорты. Такое решение сохраняет преимущества open-source подходов, устраняет их ключевые недостатки и отвечает требованиям современных команд к

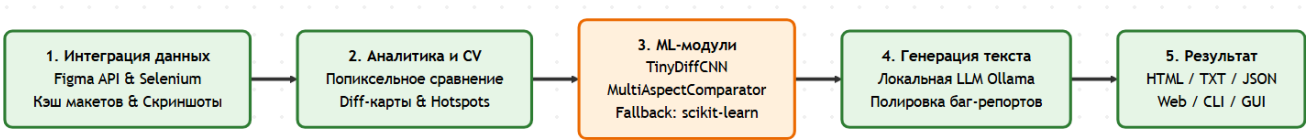
безопасности, стоимости и интерпретируемости результатов визуального регрессионного тестирования.

1.2.1 Облачные платформы визуального контроля

Развитие методологий гибкой разработки (Agile, DevOps) и практик непрерывной интеграции (CI/CD) привело к существенному увеличению частоты релизов программного обеспечения. В таких условиях традиционные подходы к визуальной приёмке интерфейсов перестали удовлетворять требованиям современных команд: ручная сверка стала узким местом, а статические скрипты на базе Selenium оказались недостаточно гибкими для отслеживания непреднамеренных визуальных изменений. Ответом на этот вызов стало появление специализированных облачных платформ визуального контроля, которые за последнее десятилетие сформировали отдельный сегмент рынка инструментов обеспечения качества (Quality Assurance).

Основные возможности облачных платформ включают автоматический сбор снимков интерфейсов в конвейере непрерывной интеграции, попиксельное сравнение с эталонными образами, наглядную визуализацию зон изменений и встроенную интеграцию с системами управления задачами. Типовая архитектура взаимодействия компонентов в SaaS-платформах визуального контроля представлена на рисунке 1.

Рисунок 1 — Типовая архитектура облачных платформ визуального контроля



Для анализа основных характеристик наиболее популярных платформ составлена таблица 1.

Таблица 1 — Сравнительная характеристика облачных платформ визуального контроля

Критерий	Percy	Applitools	Chromatic
Стоимость	от \$29/мес	от \$30/мес	от \$79/мес
Интеграция с Figma	Нет	Частично	Нет
Отчёты на русском	Нет	Нет	Нет
Локальное выполнение	Нет	Частично	Нет
Использование ИИ	Нет	Да	Нет

Среди преимуществ облачных платформ выделяются высокая стабильность работы, готовая инфраструктура для распределенных команд, поддержка различных браузерных окружений и подробная история изменений версий. Платформы предоставляют готовые интеграции с популярными системами непрерывной интеграции и позволяют масштабировать процесс тестирования на большие проекты.

К недостаткам относятся обязательная рост стоимости при увеличении частоты проверок, необходимость передачи скриншотов на внешние серверы, что противоречит требованиям конфиденциальности, а также формирование отчётов преимущественно на английском языке и отсутствие прямой интеграции с Figma.

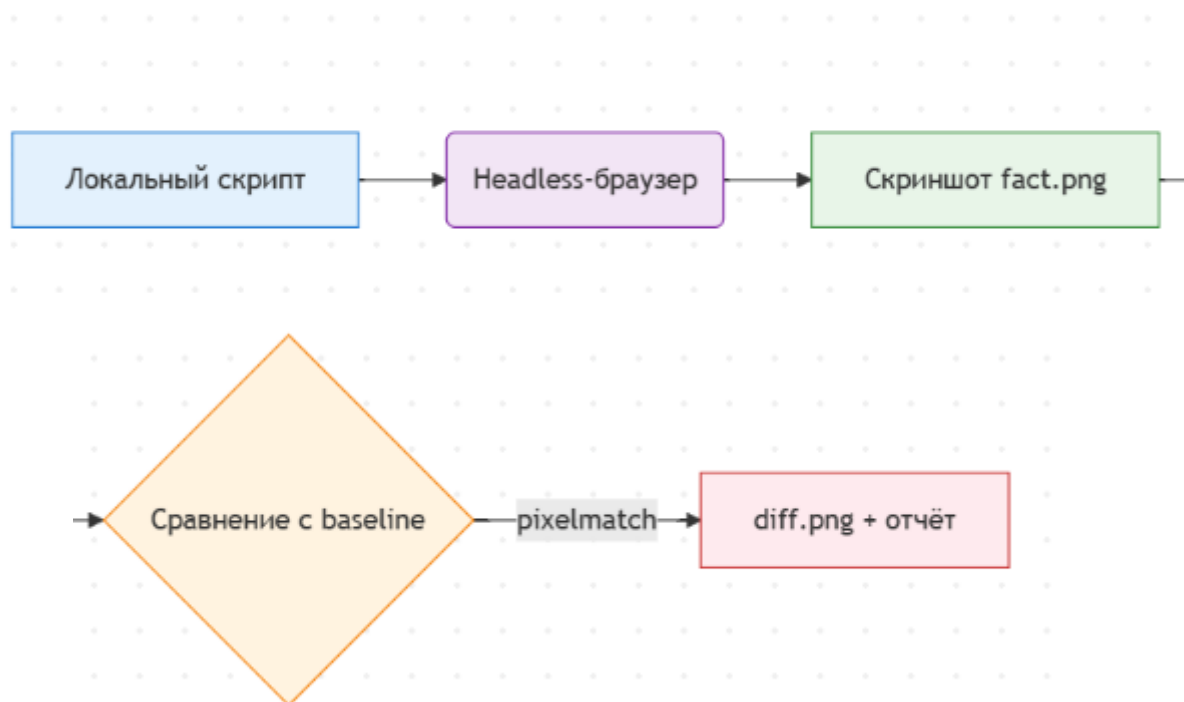
Целевой аудиторией выступают крупные продуктовые команды и предприятия с выделенным бюджетом на инфраструктуру качества.

Исходя из вышесказанного, можно сделать вывод, что облачные платформы предоставляют мощный функционал для профессионального визуального тестирования, но их финансовая нагрузка и зависимость от внешней инфраструктуры делают их недоступными для небольших студий и команд, работающих с закрытыми макетами, где требуется локальное развертывание без передачи данных третьим лицам.

1.2.2 Открытые инструменты попиксельного сравнения

BackstopJS, а также встроенные средства визуального сравнения в Playwright обеспечивают возможность локального запуска и являются бесплатными решениями. Для стека «Python + Pytest + Selenium» отмечается, что Python «очень подкупает своей доступностью и простотой», а Pytest представляет собой «простой и надёжный раннер». Ю. Кан указывает, что встроенное сравнение изображений в Playwright «бесплатно», однако не обеспечивает кроссбраузерность «из коробки». Типовая схема работы open-source инструментов визуального контроля представлена на рисунке 2.

Рисунок 2 — UML Component Diagram архитектуры системы



Ключевыми недостатками рассмотренных инструментов являются ручное сохранение эталонных изображений, отсутствие интеграции с Figma API, а также невозможность генерации семантических описаний дефектов на русском языке.

1.2.3 Ручная сверка

Традиционный метод визуальной проверки интерфейсов предполагает непосредственное участие человека в процессе сопоставления реализованной веб-страницы с утверждённым дизайн-макетом. Данный подход остаётся наиболее распространённым в небольших проектах и на начальных этапах разработки, когда автоматизация ещё не внедрена или нецелесообразна по экономическим соображениям.

Суть ручной проверки заключается в следующем: специалист открывает в браузере готовую страницу и параллельно просматривает макет в графическом редакторе, визуально сравнивая расположение элементов, их размеры, отступы, шрифты и цветовые решения. Для повышения точности используются встроенные средства разработчика в браузере, которые позволяют просматривать параметры отдельных элементов, а также режим инспекции в самом редакторе макетов, отображающий расстояния между объектами и характеристики текста.

Преимущества ручного метода очевидны: он не требует дополнительных финансовых затрат на покупку лицензий или настройку инфраструктуры, обеспечивает мгновенную обратную связь в процессе разработки и позволяет проверять отдельные компоненты с высокой точностью. Специалист может интуитивно оценить, насколько критично то или иное расхождение, и принять решение о необходимости правки без формальных процедур. Кроме того, данный подход не требует установки дополнительного программного обеспечения и работает в любой операционной среде.

Однако недостатки ручной сверки существенно ограничивают её

применимость в современных условиях. Во-первых, процесс проверки занимает значительное время, особенно при работе с многостраничными сайтами или сложными интерфейсами. Во-вторых, результат сильно зависит от внимательности и опыта конкретного человека: один специалист может пропустить важное расхождение, которое заметит другой. В-третьих, при частых обновлениях макета или регулярных релизах ручная проверка становится узким местом, замедляющим весь цикл разработки. В-четвёртых, отсутствует фиксируемый отчёт: если ошибка была пропущена при приёме, сложно установить, на каком этапе она возникла и кто её допустил. Наконец, субъективность оценки приводит к разногласиям между дизайнерами и разработчиками относительно допустимости тех или иных отклонений.

Целевой аудиторией ручного метода являются индивидуальные разработчики, фрилансеры, небольшие студии на старте проектов, а также учебные группы, где осваиваются базовые принципы вёрстки. В этих случаях объём работ невелик, а требования к масштабируемости процесса отсутствуют.

Ручная проверка остаётся полезным инструментом для точечной отладки отдельных элементов и обучения основам визуального контроля. Однако она не способна обеспечить системное, быстрое и воспроизводимое качество при активной итеративной разработке. Отсутствие автоматизации, фиксированных отчётов и объективных критериев оценки делает этот метод непригодным для проектов с частыми обновлениями, что подтверждает необходимость внедрения специализированных инструментов автоматизированной сверки.

1.2.4 Сравнительный анализ и выводы

Для систематизации результатов исследования проведённый выше анализ характеристик различных подходов к проверке интерфейсов был сведён в единую сравнительную матрицу. В таблице 2 наглядно сопоставлены недостатки существующих методов с функциональными

возможностями разрабатываемой системы.

Таблица 2 - Сравнительный анализ решений

Стоимость внедрения	Облачные платформы (SaaS)	Открытые инструменты (OSS)	Ручная проверка	Разрабатываемая система
Стоимость	Высокая (подписка)	Бесплатно	Бесплатно	Бесплатно
Хранение данных	Облако (внешние серверы)	Локально	Не хранится	Локально
Интеграция с Figma	Отсутствует или частичная	Отсутствует	Ручной экспорт	Прямая (через API)
Язык отчётов	Английский	Английский	Русский (субъективно)	Русский (субъективно)
Интеллектуальный анализ	Да (в облаке)	Нет	Да (человеческий фактор)	Да (локальная нейросеть)
Сложность настройки	Низкая	Высокая	Низкая	Средняя

Анализ представленных в таблице данных показывает, что разрабатываемая информационная система занимает уникальную нишу, объединяя преимущества, которые в существующих аналогах распределены фрагментарно. В отличие от коммерческих облачных сервисов, система

работает полностью локально и не требует ежемесячных отчислений, что критически для небольших команд с ограниченным бюджетом. При этом сохраняется уровень безопасности данных, недостижимый для SaaS-решений, поскольку изображения макетов и скриншоты не покидают компьютер пользователя.

По сравнению с открытыми инструментами попиксельного сравнения, разрабатываемая система предлагает принципиально иной уровень взаимодействия с дизайнерскими макетами. Вместо ручного сохранения эталонов и сложной настройки скриптов в ней реализован механизм прямой загрузки изображений через Figma API.

Главное отличие заключается в интерпретации результатов. Если традиционные инструменты (BackstopJS, Playwright и другие) только подсвечивают изменившиеся пиксели, то наша система с помощью локальных языковых моделей автоматически формирует понятные текстовые описания дефектов на русском языке. Например, вместо множества красных пикселей в области системного бара она выдаст: «Изменилось время в строке состояния», а при различии текста на карточке определит: «Текст на карточке товара отличается от макета, хотя структура и расположение элементов совпадают». Это существенно сокращает время на анализ отчёта и избавляет от ручной расшифровки «визуального шума».

В сопоставлении с ручной проверкой программный комплекс многократно ускоряет выполнение рутинных операций сверки и устраняет субъективный фактор на этапе первичного обнаружения расхождений. Кроме того, система стандартизирует формат фиксации ошибок, создавая единый и понятный протокол взаимодействия между дизайнерами и разработчиками, чего невозможно добиться при «глазомерном» методе.

Таким образом, проведённый анализ подтвердил наличие свободной ниши для локального инструмента визуального тестирования, который объединяет автоматизацию получения данных, методы компьютерного зрения и возможности генеративного искусственного интеллекта.

2 Проектирование системы

2.1 Постановка задачи на разработку

Целью проектного этапа является разработка системы для автоматизированного визуального сравнения веб-вёрстки с макетами Figma. Основной процесс представляет собой последовательный конвейер, включающий: автоматическую загрузку макета через Figma API, съёмку веб-страницы с помощью Selenium, построение diff-карты, анализ расхождений методами компьютерного зрения и нейросетей, а также генерацию структурированного отчёта с текстовыми описаниями дефектов.

Для достижения поставленной цели определены следующие основные подзадачи:

1. Разработка модуля автоматической выгрузки дизайн-макетов непосредственно из Figma с использованием FIGMA API;
2. Реализация съёмки веб-страниц в браузере Google Chrome с помощью инструмента Selenium WebDriver при заданных размерах viewport и условиях отображения;
3. Построение дифференциальной (diff) карты с применением методов компьютерного зрения библиотеки OpenCV;
4. Классификация и сегментация выявленных зон расхождений с использованием свёрточных нейронных сетей (CNN);
5. Фильтрация ложных срабатываний и шумовых отличий для повышения точности анализа;
6. Генерация понятных, простых описаний дефектов и их «полировка» с помощью локальной языковой модели Ollama;
7. Разработка удобных интерфейсов взаимодействия: веб-интерфейса и графического пользовательского интерфейса (GUI).

Предлагаемая архитектура строится по модульному принципу, так как это обеспечивает гибкость, масштабируемость и удобство дальнейшей поддержки системы. Каждый модуль выполняет строго определённую

функцию, а взаимодействие между ними осуществляется через унифицированные интерфейсы обмена данными.

2.2.1 Функциональные требования

Разрабатываемая система должна обеспечивать автоматизированный сбор исходных данных, включая выгрузку актуальных дизайн-макетов через официальный REST API Figma с последующим локальным кэшированием, а также программный захват скриншотов веб-страниц в контролируемой среде браузера Google Chrome с гибкой настройкой параметров viewport и задержек рендеринга. На основе полученных эталонных и фактических изображений система осуществляет построение дифференциальных карт (diff-карт) с применением настраиваемых порогов чувствительности (`pixel_threshold`, `tolerance_shift_px`), проводит глубокий анализ выявленных расхождений, интеллектуально фильтрует ложные срабатывания и формирует итоговый бинарный вердикт соответствия страницы (PASS/FAIL).

При обнаружении визуальных регрессий система обязана генерировать детализированный отчет, содержащий миниатюры проблемных фрагментов (кропы) и их структурированные описания. Ключевой функциональной особенностью является интеграция с локальной языковой моделью (платформа Ollama) для лингвистической «полировки» и формирования человекочитаемых описаний дефектов на русском языке. Результирующие данные экспортируются в форматы HTML и TXT, при этом архитектура системы должна поддерживать механизм graceful degradation, гарантирующий корректное завершение конвейера и формирование базового эвристического отчета даже в условиях временной недоступности нейросетевых компонентов.

2.2.2. Нефункциональные требования

Нефункциональные требования определяют, какими свойствами должна обладать система и в каких условиях она будет работать.

Полный цикл визуальной сверки одной веб-страницы (от загрузки макета до формирования отчёта) не должен превышать 15 минут на стандартном пользовательском оборудовании. Вся обработка изображений, анализ и генерация описаний должны выполняться локально, без передачи конфиденциальных данных на внешние серверы (за исключением авторизованных запросов к Figma API) .

Конфиденциальная информация, такая как токены доступа, должна храниться исключительно в переменных окружения. Архитектура системы должна быть модульной, что позволит в дальнейшем легко заменять отдельные компоненты (например, модель классификации или языковую модель) без переработки всего приложения.

Система должна поддерживать работу на операционных системах Windows 10 и выше, а также на популярных дистрибутивах Linux. Интерфейс управления (веб-панель и CLI) должен быть интуитивно понятным и обеспечивать мониторинг процесса в реальном времени. Особое внимание уделяется устойчивости системы к временным сбоям внешних сервисов и возможности параллельной обработки нескольких страниц.

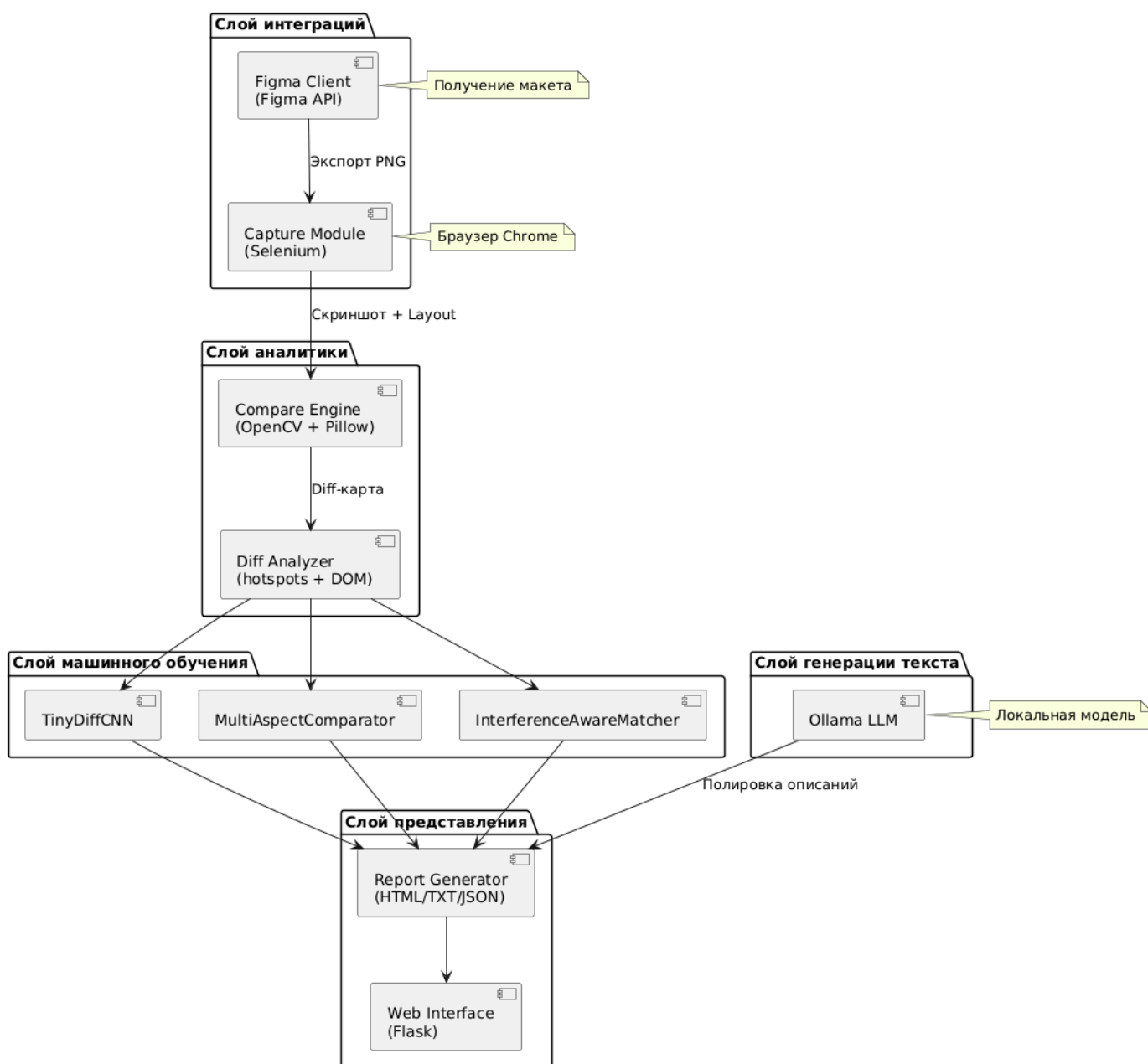
2.3 Архитектура системы

Система построена на модульной конвейерной архитектуре со строгой типизацией данных, что позволяет независимо развивать и тестировать каждый компонент. Главный принцип работы — полная локальность: все изображения, веса моделей и отчёты остаются на компьютере. Единственное внешнее обращение — это авторизованный запрос к Figma API для получения эталона

Архитектура ориентирована на полностью локальное выполнение всех

этапов обработки. Изображения макетов, скриншоты страниц, карты различий, веса нейросетевых моделей и генерируемые отчёты не передаются на внешние облачные сервисы. Общая структура системы показана на рисунке 3.

Рисунок 3 —UML Component Diagram архитектуры системы



Все компоненты работают полностью локально. Взаимодействие между модулями осуществляется через унифицированные структуры данных (RunConfig, CompareResult, BugItem).

2.4. Проектирование модульной структуры и потоков данных

Модульная структура системы спроектирована с соблюдением принципов инкапсуляции, благодаря чему каждый этап визуальной сверки обладает строго определёнными входными и выходными параметрами, а также собственным набором конфигурируемых настроек. Поток данных организован по принципу детерминированного однонаправленного конвейера: результирующий объект, сформированный на предыдущем этапе обработки, автоматически передаётся на вход следующему модулю, обеспечивая непрерывность и логическую целостность всего процесса анализа.

Этап 1. Получение эталонного изображения. На вход модуля подаются идентификаторы Figma-файла (`file_key`, `node_id`), масштаб экспорта `scale` и токен для доступа к макету `FIGMA_ACCESS_TOKEN`. Модуль `export_frame_png` выполняет запрос к REST API Figma и сохраняет полученное изображение в директорию `shots/figma_cache/`. При включённом параметре `use_cached_png = true` используется ранее сохранённая версия файла. Данный подход соответствует лучшим промышленным практикам, когда «данные забираются напрямую из Figma-файла через API».

Этап 2. Съёмка фактического состояния страницы. Модуль `capture_screenshot` открывает указанный URL в браузере Google Chrome, устанавливает заданный размер окна (`window_size`), ожидает указанное количество секунд (`capture_wait_seconds`) для завершения рендеринга динамических элементов и сохраняет скриншот. Параллельно извлекается структура страницы (`layout`) — список видимых DOM-элементов с координатами, размерами и текстовым содержимым.

Этап 3. Попиксельное сравнение изображений. Модуль `compare_screenshots` приводит оба изображения к единому размеру и строит карту различий с учётом параметров `pixl_threshold`, `tolerance_shift_px` и `tolerance_speckle_iter`. На выходе формируется объект `CompareResult`,

содержащий метрики `mse`, `changed_ratio`, `changed_ratio_raw`, `changed_ratio_shift` и путь к diff-изображению. Используемый метод соответствует классическому подходу скриншотного тестирования, при котором «изображения сравниваются пиксель к пикселю».

Этап 4. Выделение горячих зон и формирование черновика отчёта. Модули `analyze_diff_for_qa` и `build_bug_report_items` выявляют области с высокой плотностью отличий (hotspots), сопоставляют их с элементами DOM-структуры и генерируют начальные (детерминированные) описания дефектов на русском языке в формате списка `bug_items`.

Этап 5. Консолидация и фильтрация с применением машинного обучения. На данном этапе выполняется объединение дублирующихся записей, а также опциональная фильтрация ложных срабатываний с помощью модели `InterferenceAwareMatcher`. Модель `MultiAspectComparator` проводит многокритериальную оценку подозрительных фрагментов по шести аспектам (общее восприятие, текст, изображения, типографика, цвет).

Этап 6. Полировка текстовых описаний. При активации параметра `polish_bug_text` модуль `refine_bug_lines_ru` отправляет черновики описаний в локальную модель `Ollama` для улучшения формулировок. В случае недоступности модели или пустого ответа сохраняется исходный детерминированный текст.

Этап 7. Формирование итогового отчёта. Модули `write_html_report`, `append_text_report` и `write_json_sidecar` сохраняют результаты в директорию `reports/` в форматах HTML, TXT и JSON.

Итоговый вердикт PASS/FAIL определяется функцией `_verdict` на основе двух критериев: доля изменённых пикселей не превышает установленный порог `diff_threshold_pct`, а вероятность наличия дефекта по результатам ML-модели ниже 0,5.

Таблица 3 — Потоки данных между модулями конвейера

Этап	Модуль	Вход	Выход
1	figma_client	file_key, node_id, token	PNG эталона
2	capture	URL, window_size	PNG страницы, layout JSON
3	compare	2 PNG, пороги	CompareResult, diff PNG
4	diff_hotspots, bug_reports	diff, layout	список bug_items
5	comparator, fragment_match	кропы, diff	отфильтрованные bug_items
6	gemma_client	черновик, diff	полированный текст
7	report	метрики, bug_items	HTML, TXT, JSON отчёты

2.5 Проектирование форматов конфигурации, хранения артефактов и отчётности

Для управления параметрами программного комплекса спроектирован единый конфигурационный файл `config.json`. Чтение и валидация данных выполняются классом `AppConfig` в модуле `config.py`. В целях обеспечения безопасности и конфиденциальности токен доступа к API Figma исключен из конфигурационного файла и передается в систему через переменную окружения `FIGMA_ACCESS_TOKEN`. Структура конфигурации разделена на логические блоки, включающие общие параметры (адрес страницы, разрешение окна, задержка захвата и пути к артефактам), параметры интеграции с Figma (ключи файла, идентификаторы узлов, масштаб и настройки кэширования), пороги классического сравнения элементов (чувствительность яркости, допуск сдвига, итерации сглаживания и критерий неуспешного прохождения теста), а также параметры нейросетевых моделей, весов компаратора и интеграции с локальной языковой моделью Ollama. Для обеспечения воспроизводимости экспериментов и сбора обучающих данных метрики текущего прогона фиксируются в JSON-сайдкарах.

Организация файловой системы предусматривает строгое разграничение генерируемых артефактов по целевым каталогам. Кэшированные макеты Figma сохраняются в папке `shots/figma_cache/`, актуальные снимки экрана тестируемого интерфейса помещаются в `shots/`, а сформированные визуальные карты расхождений — в `shots/diffs/`. Сводная отчетность в форматах HTML и JSON накапливается в каталоге `reports/`, при этом для фиксации конкретных сессий тестирования создаются изолированные директории вида `witness` с копиями эталонов и результатов. Бинарные файлы весов нейросетей размещаются в папке `weights/`, а справочные данные, шаблоны дефектов и примеры для контекстного обучения языковой модели — в `data/bugs/`.

Интерпретация результатов автоматизированной проверки реализуется

через генерацию выходных документов двух типов. Интерактивный одностраничный HTML-отчет агрегирует общий статус проверки, метрики расхождения, предсказанную нейросетью вероятность дефекта и структурированную таблицу обнаруженных багов с кроп-изображениями несоответствий, текстовым описанием, фрагментом разметки интерфейса и ссылкой на полную дифференциальную карту. Хронологический текстовый лог выполнения дублируется в файлы с временной меткой в названии для последующей автоматической обработки в системах непрерывной интеграции. Для стандартизации формулировок дефектов применяется русскоязычный каталог эталонных шаблонов детерминированной генерации баг-репортов, а также сопутствующий файл с примерами корректного описания багов для смещения текстовой генерации языковой модели Ollama в сторону лаконичного технического стиля.

2.6. Выводы по проектной части

В ходе выполнения проектной части курсовой работы был успешно решен комплекс взаимосвязанных задач, направленных на теоретическое обоснование и техническое проектирование системы автоматизированной сверки веб-интерфейсов. На начальном этапе были четко сформулированы функциональные и нефункциональные требования к разрабатываемому программному комплексу, охватывающие весь рабочий цикл от загрузки эталонных макетов через программный интерфейс Figma API до формирования итогового отчета с детерминированными русскоязычными описаниями дефектов. Для реализации данных требований была спроектирована модульная конвейерная архитектура, разделенная на пять функциональных слоев, включающих модули интеграции, аналитики, машинного обучения, генерации текстовых описаний и представления результатов, централизованное управление которыми возложено на сквозной оркестратор.

В рамках архитектурного проектирования были детально описаны потоки данных между всеми семью последовательными этапами конвейера, начиная с экспорта графических эталонов и заканчивая архивацией изолированных копий-свидетелей сессии тестирования вместе с сопутствующими конфигурационными сайдкарами. В качестве информационной основы системы определены унифицированные форматы конфигурационных файлов, структура классов валидации параметров, топология каталогов для упорядоченного хранения бинарных и текстовых артефактов, а также композиция выходных документов.

Важнейшим результатом проектирования стало аналитическое обоснование гибридного подхода, в рамках которого сочетание детерминированных правил анализа дифференциальных карт и разметки интерфейса гарантирует стабильность базовых проверок, применение специализированных нейросетевых модулей минимизирует вероятность ложных срабатываний, а интеграция локальной языковой модели позволяет существенно повысить читаемость технических формулировок баг-репортов без нарушения контура безопасности данных. Сформированные проектные решения служат завершенной основой для последующего этапа программной реализации и обеспечивают высокий уровень модульности, автономности и гибкости конфигурирования компонентов разработанного комплекса.

3. Реализация разрабатываемой системы

3.1. Реализация конвейера получения и первичной обработки изображений

Программная реализация комплекса выполнена на языке Python. Данный стек выбран для интеграции инструментов компьютерного зрения, глубокого обучения и веб-компонентов в рамках единого вычислительного конвейера. В состав системы входят микрофреймворк Flask, библиотека Selenium для управления браузером Google Chrome, а также пакеты Pillow, NumPy и OpenCV для высокопроизводительной обработки матриц изображений.

Интеграция с графическим редактором Figma вынесена в модули `figma_client.py` и `figma_pull.py`. Функция `export_frame_png` отправляет авторизованный GET-запрос к REST API Figma по эндпоинту `/v1/images/{file_key}` с указанием идентификатора узла макета (`ids`) и масштаба (`scale`). Загруженный графический файл сохраняется в локальный кэш `shots/figma_cache/`. При повторных запусках системы и активном флаге `use_cached_png` обращения к удаленному API блокируются для снижения сетевой нагрузки. Принудительный сброс кэша управляется флагом в веб-панели или переменной `FIGMA_FORCE_REFRESH`. Токен авторизации считывается из переменной окружения `FIGMA_ACCESS_TOKEN`, и при его отсутствии выполнение прерывается с ошибкой.

Автоматизированный захват веб-страницы реализован в модуле `capture.py` с помощью Selenium WebDriver. Алгоритм включает переход по целевому URL-адресу, установку разрешения окна браузера через `window_size`, фиксированное ожидание `capture_wait_seconds` для рендеринга динамического контента и сохранение скриншота в формате PNG. Одновременно через выполнение JavaScript-сценария извлекается объектная модель разметки страницы (`layout`), содержащая координаты, размеры и

текстовое содержимое видимых элементов интерфейса.

Сравнение изображений инкапсулировано в функции `compare_screenshots` модуля `compare.py`. Метод приводит эталон и факт к RGB-формату и единому размеру. При превышении разрешения в 8 миллионов пикселей выполняется пропорциональное масштабирование вниз для защиты от нехватки оперативной памяти. Далее вычисляется попиксельная разность матриц с учетом допустимого пространственного сдвига `tolerance_shift_px`. Морфологическая операция открытия (`tolerance_speckle_iter`) подавляет одиночные шумы и артефакты сглаживания. По очищенной маске рассчитываются метрики `MSE`, `changed_ratio`, `changed_ratio_raw`, `changed_ratio_shift`, после чего `diff`-изображение с подсветкой измененных зон сохраняется в каталог `shots/diffs/`. Операции сравнения выполняются средствами `Pillow` и `NumPy` без привлечения тяжелых методов `OpenCV`.

Централизованное управление потоками данных реализовано в функции `run_pipeline` оркестратора `pipeline.py`. Конвейер выполняет обработку в следующем детерминированном порядке:

1. Экспорт эталона из Figma API или чтение файла из локального кэша (`export_frame_png`).
2. Съемка скриншота веб-страницы и извлечение геометрии элементов разметки (`capture_screenshot`).
3. Попиксельный анализ, фильтрация шумов, расчет `MSE` и генерация `diff`-карты (`compare_screenshots`).
4. Опциональное предсказание общей вероятности дефекта сверточной нейросетью `TinyDiffCNN` (`predict_fail_prob`).
5. Локализация зон наибольшего расхождения (`analyze_diff_for_qa`).
6. Формирование черновика зафиксированных отклонений разметки (`build_bug_report_items`).
7. Фильтрация и консолидация обнаруженных багов (`finalize_bug_report_items`).

8. Опциональная проверка смещенных графических элементов модулем `fragment_matcher`.
9. Опциональный запуск нейросетевой модели `MultiAspectComparator` на вырезанных подозрительных кропах.
10. Опциональное лингвистическое улучшение и перевод баг-репорта на русский язык с помощью локальной модели `Ollama`.
11. Запись интерактивного HTML-отчета, текстового лога выполнения и структурированного JSON-сайдкара.

При возникновении системных ошибок загрузки динамических библиотек `PyTorch` (например, `WinError 1114`) нейросетевой конвейер автоматически изолируется. Система не прерывает работу, а стабильно переключается на использование классических алгоритмов сравнения и попиксельных порогов яркости.

3.2. Реализация нейросетевых модулей классификации и сопоставления фрагментов

В системе реализованы три нейросетевых компонента на базе фреймворка `PyTorch`, дополняющих детерминированный попиксельный анализ. Первым является классификатор карт расхождений `TinyDiffCNN` (модуль `diff_classifier.py`), который обрабатывает фрагменты `diff`-карт размером 64 на 64 пикселя и вычисляет вероятность наличия реального дефекта верстки. Полученная оценка используется в функции вынесения вердикта совместно со стандартным пиксельным порогом. Обучение сети автоматизировано скриптом `train.py`, а при отсутствии `CUDA`-совместимого графического ускорителя активируется резервный сценарий обучения на базе библиотеки `scikit-learn` с сохранением модели в формате `joblib`.

Вторым компонентом является модель `MultiAspectComparator` (модуль `multi_aspect.py`), предназначенная для многоаспектного сравнения пар изображений. На вход нейросети подается четырехканальный тензор

размером 224 на 224 пикселя, содержащий матрицы яркости макета, сайта, их абсолютной разности и средней яркости. Архитектура базируется на предобученной предобученной сети MobileNetV3-Small с дополнительным входным адаптером каналов и промежуточным слоем с регуляризацией Dropout. На выходе модель имеет шесть независимых сигмоидальных голов, определяющих коэффициенты схожести по параметрам общего соответствия, текста, графики, расположения элементов, типографики и цвета. Модель обучена на синтетических данных и датасете Rico, достигнув на валидации среднего F1-score 0,971% при средней абсолютной ошибке 0,027%.

Третьим элементом выступает модуль сопоставления смещенных фрагментов InterferenceAwareMatcher (пакет src/fragment_match/). В отличие от стандартных энкодеров, данная архитектура сохраняет точность сопоставления в условиях визуальных помех за счет применения сверточного кодировщика с групповой нормализацией, механизма эмбедингов типов сегментов и двухслойного Transformer-энкодера. Модель обучалась на синтетических тройках фрагментов с добавлением искусственного шума и достигла точности в диапазоне 0,76%–0,81%. В общем конвейере этот компонент фильтрует пункты баг-репорта, отсекая ложные срабатывания в зонах, где зафиксировано идентичное графическое содержимое при наличии пространственного сдвига.

Нейросетевой анализ дополняется группой эвристических скриптов. Модуль section_compare.py отвечает за изолированную проверку логотипов, эмодзи и статистических блоков интерфейса. Компонент typography_compare.py верифицирует параметры шрифтов и цветов на основе вычисленных стилей объектной модели веб-страницы. Для сопоставления текстового содержимого элементов разметки с графическим эталоном на кропах макета применяются инструменты оптического распознавания EasyOCR и Tesseract.

3.3. Интеграция локальной языковой модели и генерация текстовых отчётов

Взаимодействие с локальным сервером Ollama организовано по протоколу REST API в модуле `gemma_client.py`. Использование данной платформы позволяет развертывать большие языковые модели непосредственно на локальном узле без отправки конфиденциальных данных во внешние облачные сервисы, что полностью закрывает требования к безопасности корпоративных и учебных проектов, работающих с закрытыми макетами.

В зависимости от установленного в конфигурации параметра `bug_report_mode`, система способна функционировать в трех режимах генерации. В режиме `text` выполняется лингвистическое перефразирование детерминированного черновика, полученного от базового модуля анализа разметки, что обеспечивает наибольшую стабильность результатов. Режим `vision` активирует прямую обработку изображений дифференциальных карт мультимодальными моделями. Комбинированный режим `both` совмещает первичный визуальный анализ с последующей текстовой полировкой формулировок.

Формирование запросов к модели инкапсулировано в модуле `bug_report_prompts.py`, который динамически подгружает примеры контекстного обучения из файла `ollama_fewshot_ru.json`. Первоначальная настройка и развертывание кастомной языковой модели с зашитым системным промптом, ориентированным на генерацию лаконичного русскоязычного QA-стиля, автоматизированы с помощью инсталляционного скрипта `setup_ollama_bug_reporter.py`.

Для обеспечения высокой отказоустойчивости при интеграции с языковыми моделями в коде реализован комплекс защитных механизмов:

1. автоматическое сопоставление сконфигурированного имени модели с

локально доступными тегами;

2. механизм повторных отправлений POST-запросов (до четырех итераций) при сетевых сбоях;
3. пропорциональное сжатие входных дифференциальных карт до пятисот двенадцати пикселей по длинной стороне для экономии оперативной памяти видеокарты;
4. жесткая фиксация параметра температуры генерации на значении ноль для обеспечения детерминированности ответов;
5. автоматический откат к эвристическому черновику при получении пустого ответа или сбое сервиса Ollama;
6. программный метод `sanitize_bug_lines` для очистки выходной строки от языковых артефактов и некорректных символов.

После завершения этапа текстовой оптимизации финальные формулировки дефектов интегрируются в общую структуру данных через функцию слияния. Это гарантирует, что итоговый интерактивный HTML-отчет и хронологический текстовый лог будут содержать улучшенные языковой моделью описания багов вместо сырых эвристических формулировок.

3.4. Реализация веб-интерфейса управления и API взаимодействия

Программный комплекс поддерживает три независимых интерфейса для запуска процесса верификации интерфейсов, ориентированных на различные сценарии эксплуатации системы.

Первым является интерфейс командной строки, реализованный в исполняемом скрипте `run_tests.py` и предназначенный для проведения однократных сессий тестирования. Данный инструмент поддерживает гибкое конфигурирование параметров запуска с помощью системы флагов. В частности, аргументы `--no-gemma` и `--no-comparator` позволяют временно

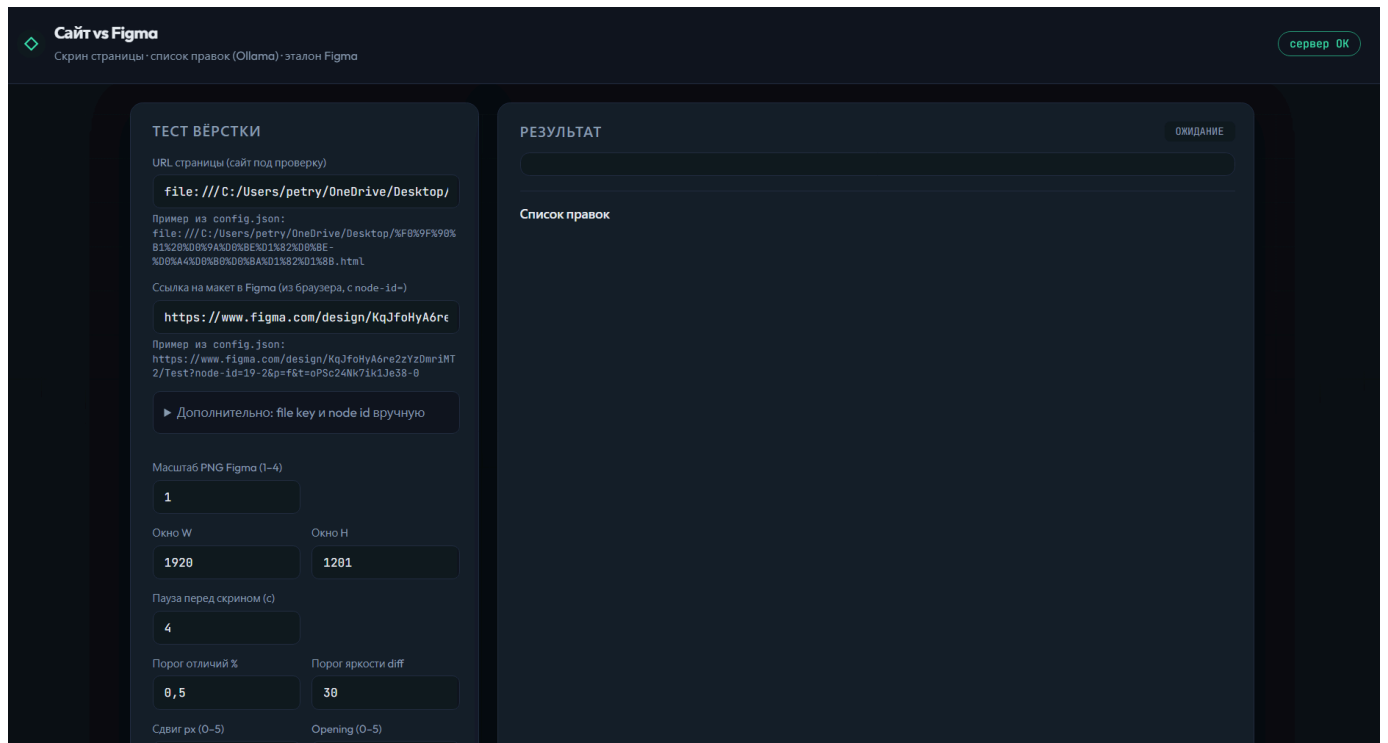
деактивировать модули Ollama и MultiAspectComparator соответственно, а флаг `--url` обеспечивает переопределение целевого адреса веб-страницы без необходимости внесения изменений в основной файл конфигурации.

Вторым интерфейсом выступает интерактивная веб-панель на базе микрофреймворка Flask, развертываемая модулем `web_server.py`. Архитектура серверной части предоставляет REST API для полноценного управления жизненным циклом тестов. Корневой маршрут обеспечивает отдачу статической HTML-формы, а специализированные эндпоинты отвечают за верификацию версии сборки и автоматическое заполнение полей веб-формы актуальными значениями из конфигурационного файла.

Процесс сверки инициализируется посредством отправки POST-запроса на соответствующий маршрут, который возвращает HTTP-статус 202 и уникальный идентификатор задачи. Сама процедура анализа выполняется асинхронно в изолированном фоновом потоке, что предотвращает блокировку сетевых ответов сервера. Мониторинг состояния запущенных задач, чтение логов в реальном времени и получение ссылок на сформированные документы осуществляются через динамический GET-маршрут. Для обеспечения потокобезопасности состояние всех активных процессов инкапсулировано в глобальном словаре, доступ к которому регулируется межпоточным блокировщиком.

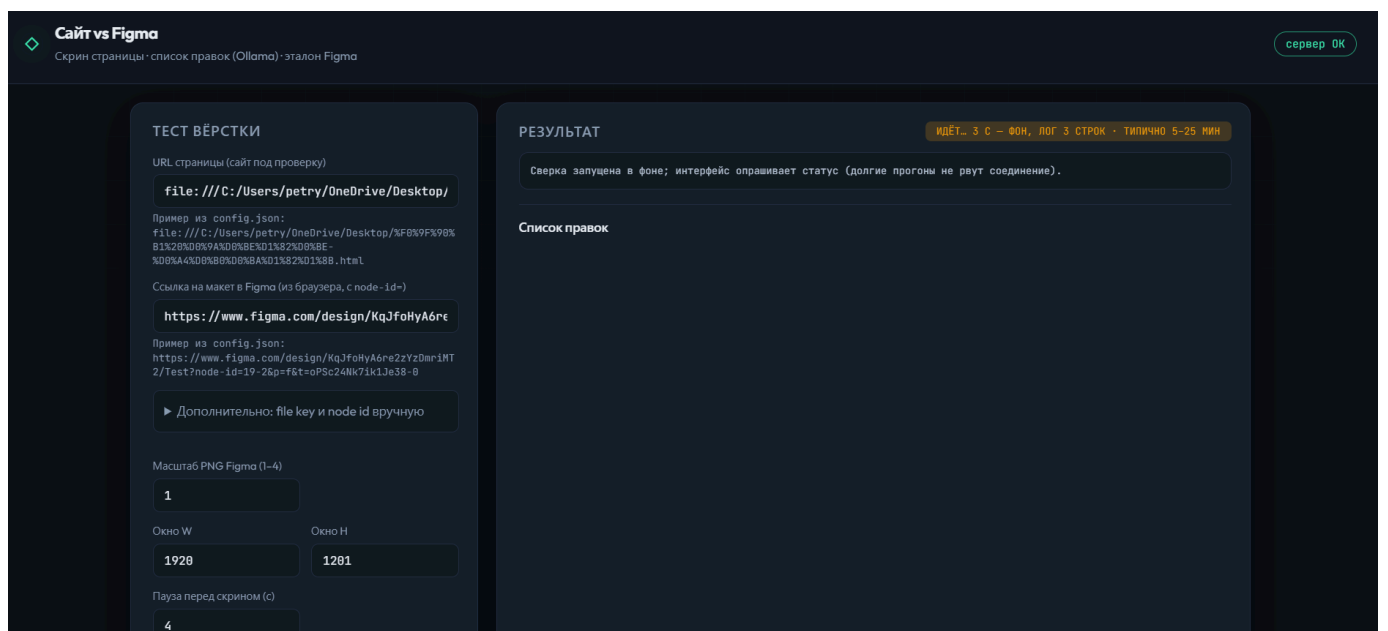
На рисунке 4 показан веб-интерфейс конфигурации параметров теста, где пользователь может задать целевой URL, разрешение viewport, пороги алгоритмов сравнения, а также селективно активировать или деактивировать нейросетевые компоненты системы.

Рисунок 4 — Веб-интерфейс конфигурации параметров теста



Процесс выполнения теста демонстрируется на рисунке 5, где в реальном времени отображаются индикатор прогресса, лог выполняемых операций и текущий статус задачи.

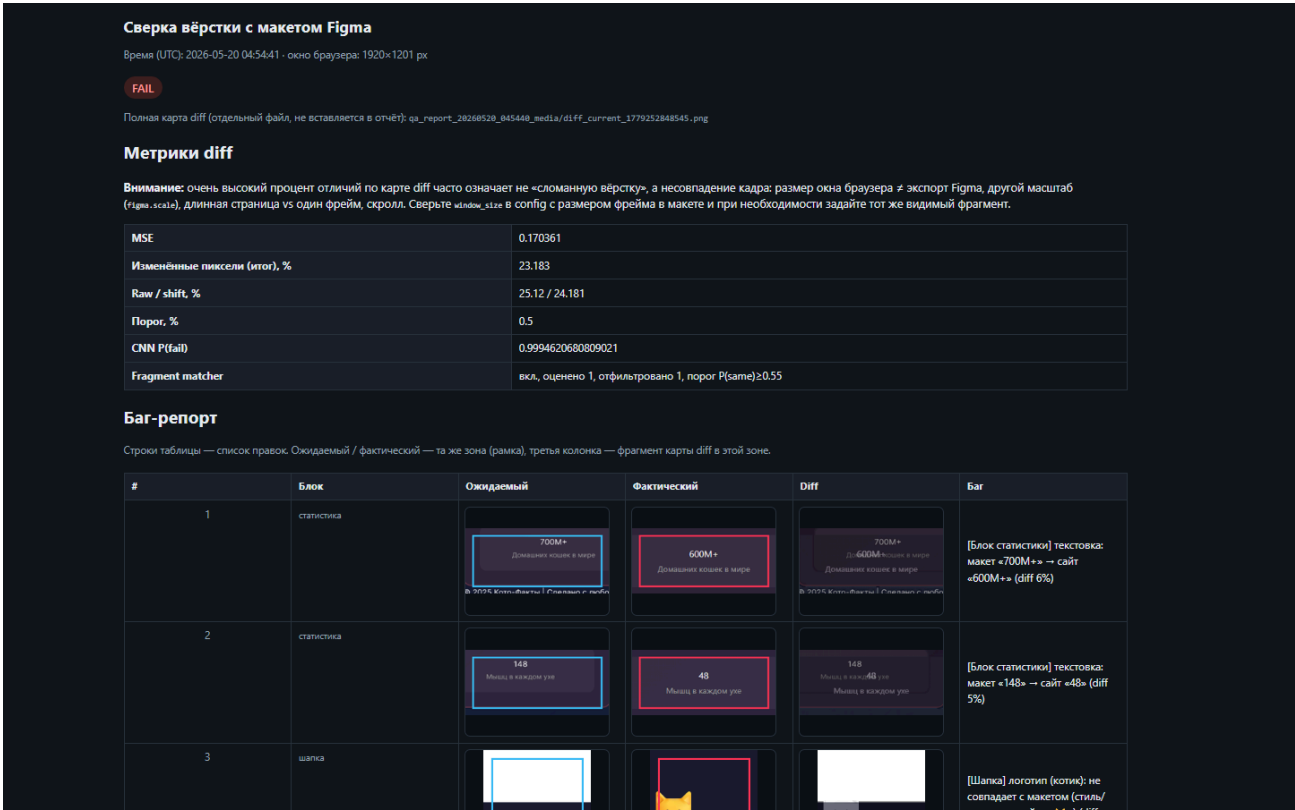
Рисунок 5 — Веб-интерфейс отслеживания прогресса теста



Результаты автоматизированной сверки представлены на рисунке 6, который содержит итоговый вердикт (PASS/FAIL), таблицу метрик (MSE,

changed_ratio), миниатюры обнаруженных дефектов с кропами проблемных зон, а также ссылки на дифференциальную карту и кнопки скачивания отчётов в форматах HTML, JSON и TXT.

Рисунок 6 - Веб-интерфейс результата работы



Интегрированная подсистема генерации отчетности, размещенная в модуле report.py, отвечает за сведение результатов работы конвейера. Функция записи формирует компактный одностраничный HTML-документ, содержащий итоговый вердикт проверки, структурированные таблицы метрик и обнаруженных дефектов. С целью оптимизации дискового пространства и повышения читаемости отчета в таблицу багов встраиваются только локальные миниатюры кроп-изображений проблемных зон без дублирования полноразмерных снимков экрана. Дополнительно в структуру документа помещаются фрагменты карты разметки элементов интерфейса и прямая ссылка на результирующую дифференциальную карту.

3.5. Выводы по реализационной части

В ходе выполнения третьей главы курсовой работы была осуществлена полная программная реализация всех компонентов системы автоматизированной сверки верстки, спроектированных на предыдущих этапах исследования. Было обеспечено построение устойчивого вычислительного конвейера на языке Python, реализующего сквозной цикл обработки данных от авторизованного импорта эталонных изображений через REST API Figma до сохранения консолидированных многоформатных отчетов.

В разработанный конвейер успешно интегрированы три специализированных нейросетевых компонента, обеспечивающих классификацию дифференциальных карт, многоаспектный анализ визуального сходства графических фрагментов и интеллектуальную фильтрацию ложных срабатываний в зонах пространственного сдвига элементов. Путем интеграции локального сервера Ollama в систему внедрены функции семантического анализа и лингвистической оптимизации формулировок баг-репортов на русском языке, функционирующие в нескольких режимах работы в закрытом контуре безопасности.

Созданная инфраструктура предоставляет конечные точки входа для трех независимых интерфейсов взаимодействия, что делает программный комплекс удобным для автоматизации тестирования, ручного управления и проведения демонстраций. Спроектированные механизмы деградации функционала и автоматического переключения на эвристические алгоритмы при сбоях в среде выполнения PyTorch или недоступности языковой модели гарантируют высокую отказоустойчивость системы. Архитектура программного комплекса ориентирована на полностью локальное развертывание в современных операционных системах и подтвердила свою работоспособность в ходе испытаний на тестовом стенде. Полученные результаты программной реализации полностью подтверждают

достижимость поставленной цели и практическую ценность разработанного инструментария.

3.6. Пример работы системы

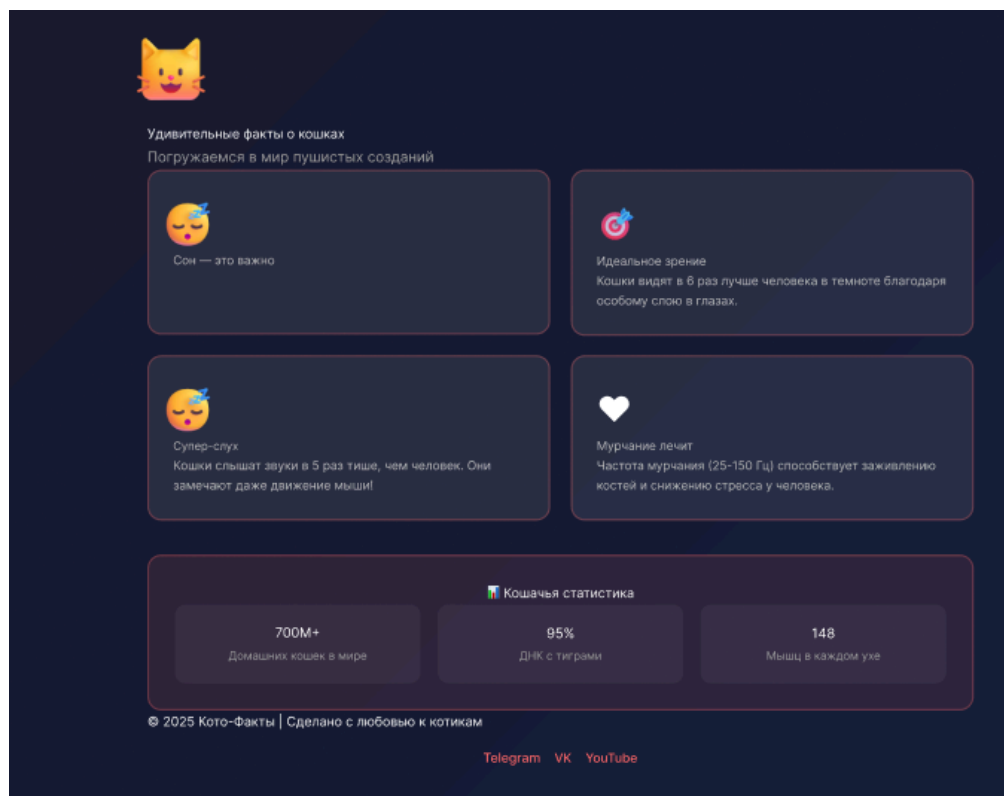
Для проверки работоспособности разработанной системы был проведён натурный эксперимент. В качестве тестовых данных использовался реальный дизайн-макет главной страницы интернет-магазина из Figma и соответствующая HTML-вёрстка.

Система выполнила полный конвейер обработки: автоматическую загрузку макета через Figma API, съёмку страницы, попиксельный анализ, нейросетевую классификацию и генерацию описаний дефектов с помощью Ollama.

Результаты работы системы представлены ниже.

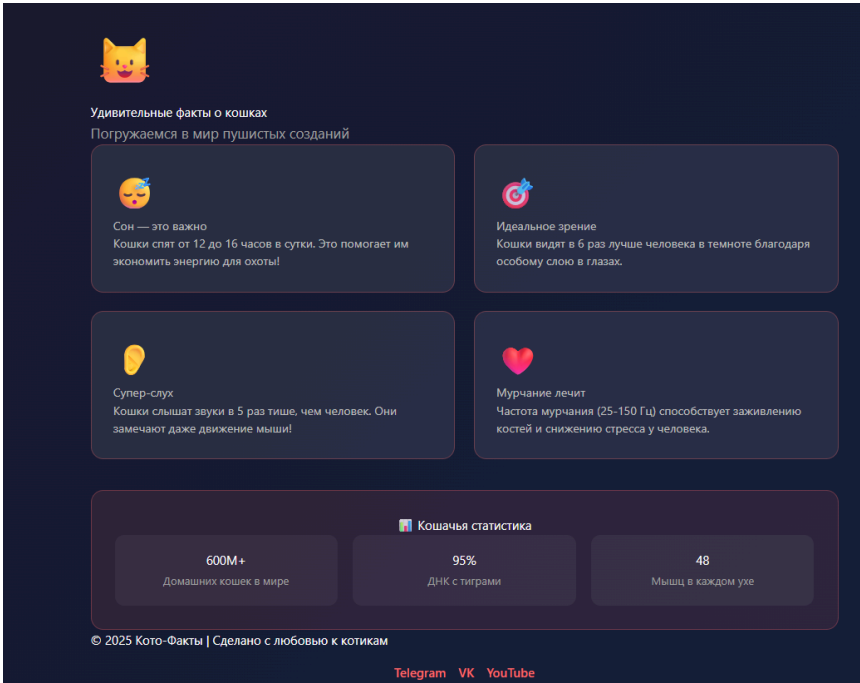
На рисунке 7 показан эталонный дизайн-макет, полученный непосредственно из Figma.

Рисунок 7 — Эталонный дизайн-макет главной страницы (Figma)



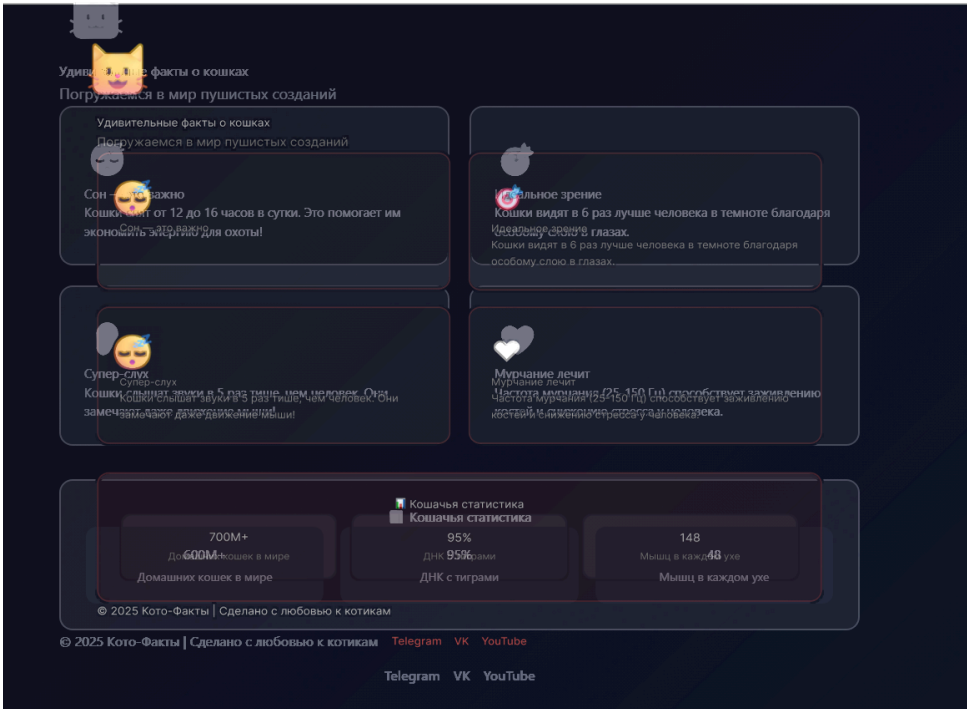
На рисунке 8 представлен актуальный скриншот реализованной веб-страницы, сделанный в браузере Google Chrome при заданном viewport.

Рисунок 8 — Скриншот реализованной вёрстки



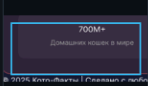
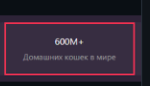
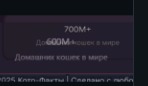
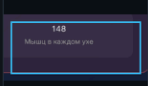
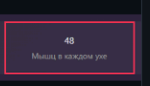
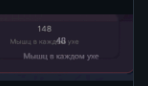
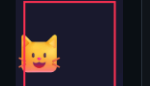
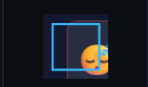
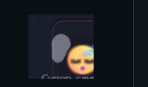
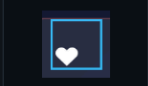
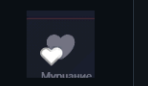
Для визуализации различий система построила дифференциальную карту (diff-карту). На рисунке 9 красным и жёлтым цветом выделены области расхождений между макетом и вёрсткой.

Рисунок 9 — Diff-карта визуальных различий



После анализа изображений и применения нейросетевых моделей (TinyDiffCNN и MultiAspectComparator) система сгенерировала структурированный отчёт. Фрагмент отчёта с описаниями дефектов, созданными локальной языковой моделью Ollama, представлен в рисунке 10.

Рисунок 10 — Пример автоматически сгенерированных описаний дефектов

Баг-репорт					
Строки таблицы — список правок. Ожидаемый / фактический — та же зона (рамка), третья колонка — фрагмент карты diff в этой зоне.					
#	Блок	Ожидаемый	Фактический	Diff	Баг
1	статистика				[Блок статистики] текстовка: макет «700M+» → сайт «600M+» (diff 6%)
2	статистика				[Блок статистики] текстовка: макет «148» → сайт «48» (diff 5%)
3	шапка				[Шапка] логотип (котик): не совпадает с макетом (стиль/иконка на сайте «🐱») (diff 46%)
4	карточка 3				[Карточка 3: 🐱 Супер-слух Кошки слышат звуки в 5...] эмодзи не совпадает на сайте «🐱» на макете другой символ (diff 42%)
5	карточка 4				[Карточка 4: 🐱 Мурчание лечит Частота мурчания...] эмодзи/иконка: другой стиль или цвет (на сайте «🐱») (diff 42%)

ЗАКЛЮЧЕНИЕ

В ходе работы была успешно реализована и протестирована система, способная автоматически получать макет из Figma, выполнять гибридный анализ и генерировать понятные русскоязычные отчёты. Приведённый в разделе 3.6 пример подтверждает достижение поставленной цели.

В рамках теоретической и проектной частей исследования был проведён подробный аналитический обзор существующих аналогов и подходов к автоматизации тестирования интерфейсов, выявивший необходимость создания гибкого локального инструмента, устойчивого к ложным срабатываниям. На основе сформированных функциональных требований была спроектирована и детализирована пятислойная модульная архитектура системы и описаны сквозные потоки данных между этапами конвейера. В ходе программной реализации на языке Python были разработаны модули асинхронного захвата интерфейса, локального кэширования эталонов и три нейросетевых компонента, среди которых модель многоаспектного сравнения фрагментов MultiAspectComparator достигла высокого показателя точности со средним F1-score на уровне 0,971 % при минимальной средней абсолютной ошибке. Проведённая экспериментальная оценка на тестовом стенде подтвердила высокую отказоустойчивость разработанных алгоритмов деградации функционала и способность системы эффективно изолировать шумы.

В качестве перспектив дальнейшего развития разработанного программного комплекса рассматривается существенное расширение объёма обучающей выборки для повышения точности сопоставления фрагментов в условиях сложных интерфейсных помех, разработка дополнительных коннекторов для автоматического экспорта сформированных языковой моделью дефектов во внешние корпоративные трекеры задач, такие как Jira и YouTrack.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Попов, П. Скриншотное тестирование: практические советы из реального проекта [Электронный ресурс] / П. Попов // Habr : [сайт]. — URL: <https://habr.com/ru/companies/ispring/articles/1025676/> (дата обращения: 24.05.2026). — Текст : электронный.
2. Тестирование Frontend [Электронный ресурс] // Habr : [сайт]. — URL: <https://habr.com/ru/articles/975252/> (дата обращения: 24.05.2026). — Текст : электронный.
3. Кабалкина, А. Figma to Frontend: как мы автоматически синхронизируем дизайн и код [Электронный ресурс] / А. Кабалкина // Habr : [сайт]. — URL: <https://habr.com/ru/companies/vk/articles/790686/> (дата обращения: 24.05.2026). — Текст : электронный.
4. Кан, Ю. Визуальное регрессионное тестирование [Электронный ресурс] / Ю. Кан // Yuri Kan — автоматизация тестирования : [сайт]. — URL: <https://yrkan.com/ru/course/module-08-automation/visual-regression-testing/> (дата обращения: 24.05.2026). — Текст : электронный.
5. Коптелов, А. Внедрение ИИ-агента в бизнес-процесс за один день: от развертывания до прототипа [Электронный ресурс] / А. Коптелов // Habr : [сайт]. — URL: <https://habr.com/ru/companies/otus/articles/1033114/> (дата обращения: 24.05.2026). — Текст : электронный.
6. Figma to Frontend: как мы автоматически синхронизируем дизайн и код [Электронный ресурс] // Habr : [сайт]. — URL: <https://habr.com/ru/companies/vk/articles/790686/> (дата обращения: 24.05.2026). — Текст : электронный.